

Python Data Types and Numerical Representations:

Python is dynamically typed $\rightarrow$ variable types determined at runtime

Basic Built in Types:

- int, float, string, file IO etc...

Numpy:

- numpy arrays: fixed type and size, efficient memory layout.

Example: `np.uint8` $\in [0, 255]$

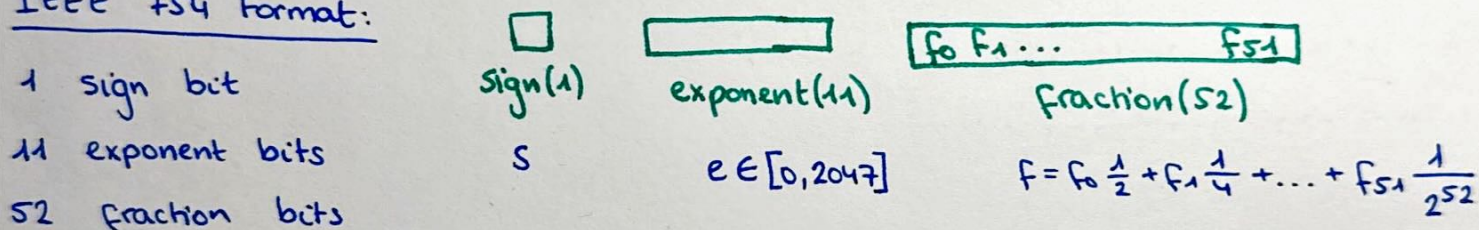
Limitation: NumPy does not work directly with JSON (must convert to lists)

Floating Point Numbers:

Python uses IEEE 754 double-precision floats

Never test equality of floats $\rightarrow$ rounding errors accumulate

Example: $0.1 + 0.2 - 0.3 \neq 0$ ($= 5.55 \times 10^{-17}$)

IEEE 754 Format:

$$\text{number} = (-1)^s \cdot (1+f) \cdot 2^{e-1023}$$

Array types:Lists

- + variable length
- + mixed types

- no bulk operation
- wasteful memory usage

Dictionaries

- + variable length
- + mixed types
- + fast key lookup

- no bulk operation
- wasteful memory usage

Numpy Arrays

- + efficient memory
- + supports vectorized operations
- + supports reshape, slice
- + best choice for numerical computations
- + fixed data type

- fixed size arrays
- no mixed types

append $\Rightarrow$ not efficient

Example: Randomly change elements in a large array

$N = 10^6$ array

Probability $p = 10^{-5}$

Method 1: Python loop (C type-like) $\Rightarrow$ Time $\sim 270\text{ms}$ (N random number for loop)

```
for c in range(N):  
    if np.random.random() < p:  
        a[c] = ...
```

Method 2: Numpy Vectorization (fast) $\Rightarrow \sim 6\text{ms}$ (N random numbers)

```
r = np.random.random(N) < p  
a[r] = ...
```

Method 3: Binomial + random choice (fastest) $\Rightarrow \sim 0.27\text{ms}$

Expected number of changes
 $N \cdot p = 10$

```
n = np.random.binom(N, p)  
ind = np.random.choice(N, n)  
a[ind] = ...
```


Linear Algebra:Permutations: each of the possible ways a set can be ordered1 2 31 3 22 1 32 3 13 1 23 2 1 $\Rightarrow$ Generating permutations (recursive idea)

1. fix each possible element in position 1

2. recursively permute the rest.

 $\Rightarrow$ produces all $n!$ permutations.Levi-Civita symbol (ϵ -tensor):General definition (n dim):

$$\epsilon_{a_1, a_2, \dots, a_n} = \begin{cases} +1 & \text{if } (a_1, a_2, \dots, a_n) \text{ is an even permutation of } (1, \dots, n) \\ -1 & \text{if } (a_1, a_2, \dots, a_n) \text{ is an odd permutation of } (1, \dots, n) \\ 0 & \text{if any two indices repeat} \end{cases}$$

3D special case:

$$\epsilon_{ijk} = \begin{cases} +1 & : (1, 2, 3), (2, 3, 1), (3, 1, 2) \\ -1 & : (2, 1, 3), (1, 3, 2), (3, 2, 1) \\ 0 & : \text{if any index repeats eg. } (1, 1, 2) \end{cases}$$

Properties:

- completely antisymmetric $\epsilon_{ijk} = -\epsilon_{jik}$
- multiplies nicely with matrices/vectors
- useful for determinants and cross products

Determinant:

$$\det(\underline{A}) = \sum_{a_1, \dots, a_n} \epsilon_{a_1, \dots, a_n} A_{1a_1} A_{2a_2} \dots A_{na_n}$$

- Levi-Civita picks out one element from each row and column

- sign is determined by whether the column order is an even or odd permutation.

$\Rightarrow$ For 3×3 this formula reduces to rule of Sarrus

calculate $\sum a_1, \dots, a_n$:

- loop through positions i
- swap with the position where digit i is
- count the swaps

Cross Product: $\underline{c} = \underline{a} \times \underline{b} \Rightarrow c_i = \sum_{jk} \epsilon_{ijk} a_j b_k$

Expanded vector form:

$$\underline{a} \times \underline{b} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}$$

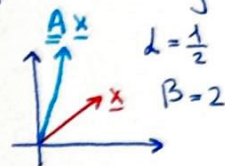
- orthogonal to both $\underline{a}$ and $\underline{b}$
- magnitude = area of parallelogram
- changes sign if you swap $\underline{a}$ and $\underline{b}$

Linear Transformations:

function $A: U \rightarrow V$ is linear if: $A(\lambda u) = \lambda A(u)$, $A(u+v) = A(u) + A(v)$

$\hookrightarrow$ any linear transformation in finite dim. space can be represented by a matrix $U=V=\mathbb{R}^n$

$$\underline{A} \underline{x} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} a_{11}x_1 + a_{12}x_2 \\ a_{21}x_1 + a_{22}x_2 \end{pmatrix}$$

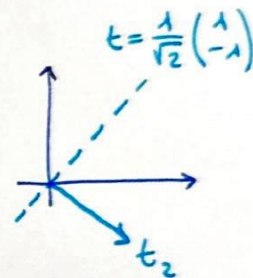


a) Scaling: $\underline{A} = \begin{pmatrix} \alpha & 0 \\ 0 & \beta \end{pmatrix} \Rightarrow \underline{A} \underline{x} = \begin{pmatrix} \alpha x \\ \beta y \end{pmatrix}$

b) Projection: $\underline{t} = \begin{pmatrix} t_1 \\ t_2 \end{pmatrix}$, $\|\underline{t}\|=1 \Rightarrow \underline{P} = \underline{t} \underline{t}^T = \begin{pmatrix} t_1 t_1 & t_1 t_2 \\ t_2 t_1 & t_2 t_2 \end{pmatrix}$

$$\underline{P} = \frac{1}{2} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad \underline{t} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Prop: $\det(\underline{P}) = 0$



c) Rotation: $\underline{R}(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$, $\det(\underline{R}) = 1$

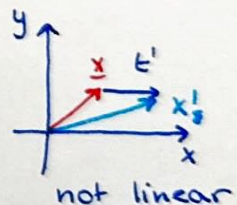
Combination of two linear operations $\Rightarrow$ product of matrix NOT Commutative

Proj $\rightarrow$ rot

Rot $\rightarrow$ proj

$\bigcirc \quad \bigcirc$

$\bigcirc \times \quad \bigcirc$



Affine Transformation:

linear part + translation

$\underline{x}$ original, $\underline{x}'$ translated $\underline{t} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} t_1 \\ t_2 \end{pmatrix}$

$$\underline{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad \underline{x}' = \begin{pmatrix} x_1 + t_1 \\ x_2 + t_2 \end{pmatrix}$$

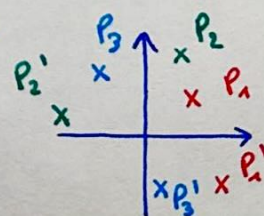
$$\begin{pmatrix} 1 & 0 & t_1 \\ 0 & 1 & t_2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix} = \begin{pmatrix} x_1 + t_1 \\ x_2 + t_2 \\ 1 \end{pmatrix} \quad (\underline{A} \underline{t}) \cdot \underline{x} \Rightarrow \text{Affine transformation}$$

$$\begin{pmatrix} \underline{A} & \underline{t} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11}x_1 + a_{12}x_2 + t_1 \\ a_{21}x_1 + a_{22}x_2 + t_2 \\ 1 \end{pmatrix}$$

6 unknowns (degrees of freedom)

$a_{11}, a_{12}, a_{21}, a_{22}, t_1, t_2$

$\Rightarrow$ give 3 point pairs



$\Rightarrow$ 3 pairs of corresponding points determine any affine transformation uniquely

Solution of systems of Linear equations:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

$$\vdots$$

$$a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n$$

$$\underline{A} \underline{x} = \underline{b}$$

$$\underline{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \Rightarrow \text{variables}$$

$\underline{A}$ coefficient matrix a_{ij} terms

$$\underline{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \Rightarrow \text{constants}$$

For given $\underline{A}$ and $\underline{b}$ what is $\underline{x}$?

construct augmented matrix $(\underline{A} | \underline{b}) \Rightarrow$ i) make upper triangular (values below diagonal zero)
ii) diagonal 1 (optional)

Legal Moves:

- swap rows
- scaling of rows (multiply with nonzero number)
- add multiple of one row to another.

$$\left. \begin{array}{l} x+y+z=6 \\ 2x-y+z=3 \\ x+2y-z=2 \end{array} \right\} \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & -1 & 1 & 3 \\ 1 & 2 & -1 & 2 \end{array} \right)$$

1. eliminate below a_{11}
2. eliminate below a_{22}

$$r_2 \rightarrow r_2 - 2r_1$$

$$r_3 \rightarrow r_3 - r_1$$

$$\sim \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & -3 & -1 & -9 \\ 0 & 1 & 2 & -4 \end{array} \right) \sim \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -2 & -4 \\ 0 & 0 & -7 & -21 \end{array} \right) \sim \left(\begin{array}{ccc|c} \boxed{1} & 1 & 1 & 6 \\ 0 & \boxed{1} & -2 & -4 \\ 0 & 0 & \boxed{1} & 3 \end{array} \right)$$

3. back subst.

$$\sim \left(\begin{array}{ccc|c} 1 & 1 & 0 & 3 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array} \right) \sim \left(\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array} \right) \quad \begin{array}{l} x=1 \\ y=2 \\ z=3 \end{array}$$

Pseudo code for Gauss Elimination (GE):

for i in $1 \dots n-1$: # loop over columns

find nonzero in $A[i,i]$ or below $A[i,i]$

swap if needed

for j in $i+1 \dots n$: # loop over rows

by subtracting the multiples of row i eliminate $A[j,i]$

GE gives upper triangular matrix. How to reconstruct $\underline{x}$?

Backward substitution: x_i can be calculated from $x_{i+1}, x_{i+2}, \dots, x_n$

do it recursively $\underline{U} \underline{x} = \underline{b}'$

x_i can be expressed as: $u_{ii} x_i = b'_i - u_{i,i+1} x_{i+1} - u_{i,i+2} x_{i+2} - \dots - u_{in} x_n$

↓
needed if diagonal not 1.

↖ $u_{nn} x_n = b'_n$

if we start $\underline{x} = 0$: $x_i = b'_i - \underline{u}_i \cdot \underline{x}$

↖ i^{th} row of $\underline{U}$

why backwards substitution?

⇒ computational cost

GE (to upper triangular) $\mathcal{O}(\frac{2}{3}n^3)$
back-wards substitution $\mathcal{O}(n^2)$ } faster

GE (for diagonal) $\mathcal{O}(\frac{4}{3}n^3)$

Note: we solve $\underline{A} \underline{x} = \underline{b}$ without calculating A^{-1} $\underline{x} = |A^{-1}| \underline{b}$

Gauss Elimination: $\underline{A} \underline{x} = \underline{b}$ $\underline{x} = ?$

- multiply row with nonzero number
- add a row to another
- swap rows, (zero or too small number in the diagonal)

Pros:

- + general
- + simple algorithm
- + exact result

Cons:

- computationally expensive $\mathcal{O}(n^3)$
- instability major issue with why its not used, (continue calculation with wrong ... 10^{-15} values)
- inefficient for sparse matrix.

Swap Rows: Pivoting:

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad r_1 \leftrightarrow r_2$$

start from $\underline{P} = \underline{I}$ (do swapping also in identity so you know which ones were swapped)

LU decomposition: already includes pivoting matrix

$$\underline{P} \underline{A} = \underline{L} \cdot \underline{U}$$

Iterative Method: Gauss-Seidel

$$x_i^{(t)} \xrightarrow{\text{iteration}} x_i^{(t+1)} \quad \text{closer to solution}$$

Line i : $a_{i1}x_1 + a_{i2}x_2 + \dots + a_{in}x_n = b_i$ Express x_1 :

$$\boxed{i=1} \quad x_1^{(t+1)} = -\frac{1}{a_{11}} (a_{12}x_2^{(t)} + \dots + a_{1n}x_n^{(t)} - b_1)$$

(based on conditions)
it is a stable fix point

$$x_2^{(t+1)} = -\frac{1}{a_{22}} (a_{21}x_1^{(t)} + \dots + a_{2n}x_n^{(t)} - b_2)$$

use new value instead of old one
 $\Rightarrow$ implicit method

$$\vdots$$

$$x_n^{(t+1)} = -\frac{1}{a_{nn}} (a_{n1}x_1^{(t+1)} + \dots + a_{n,n-1}x_{n-1}^{(t+1)} - b_n)$$

- generally implicit methods (where newly calculated partial solution is used) are better than explicit ones.

$$x_i^{(t+1)} = -\frac{1}{a_{ii}} \left(\sum_{j=1}^{i-1} a_{ij} x_j^{(t+1)} + \sum_{j=i+1}^n a_{ij} x_j^{(t)} - b_i \right)$$

Condition for convergence:

- diagonally dominant $|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|$
 - positive definite
 - symmetric
- } Hermitian $\underline{\underline{A}} = \underline{\underline{A}}^H$ (complex conjugate transpose)

Eigenvalues, Eigenvectors:

Standard method:

- eigenvalues: roots of characteristic polynomial
- eigenvectors: solve a set of singular linear eqn.

$$\underline{\underline{A}} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \quad \text{characteristic polynomial: } 0 = \det(\underline{\underline{A}} - \lambda \underline{\underline{I}}) = \begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = \lambda^2 - 4\lambda + 3$$

$$\lambda_1 = 1, \quad \lambda_2 = 3$$

$$\underline{\underline{V}}_1: (\underline{\underline{A}} - \lambda_1 \underline{\underline{I}}) \underline{\underline{V}}_1 = 0 \quad \left(\begin{array}{cc|c} 1 & 1 & 0 \\ 1 & 1 & 0 \end{array} \right) \sim \left(\begin{array}{cc|c} 1 & 1 & 0 \\ 0 & 0 & 0 \end{array} \right) \quad \underline{\underline{V}}_1 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$\underline{\underline{V}}_2: (\underline{\underline{A}} - \lambda_2 \underline{\underline{I}}) \underline{\underline{V}}_2 = 0 \quad \left(\begin{array}{cc|c} -1 & 1 & 0 \\ 1 & -1 & 0 \end{array} \right) \sim \left(\begin{array}{cc|c} 1 & -1 & 0 \\ 0 & 0 & 0 \end{array} \right) \quad \underline{\underline{V}}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Largest Eigenvalue:

$\underline{\underline{A}}(n \times n)$, eigenvalues are $\lambda_1, \dots, \lambda_n$ such that $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$

$\underline{\underline{V}}_1 \quad \underline{\underline{V}}_2 \quad \underline{\underline{V}}_n$

$$\underline{\underline{x}}_0 \in \mathbb{R}^n \quad \underline{\underline{A}} \underline{\underline{x}}_0 = ? \quad \underline{\underline{x}}_0 = c_1 \underline{\underline{V}}_1 + \dots + c_n \underline{\underline{V}}_n$$

$$\underline{\underline{A}} \underline{\underline{x}}_0 = \underline{\underline{A}} (c_1 \underline{\underline{V}}_1 + \dots + c_n \underline{\underline{V}}_n) = \lambda_1 c_1 \underline{\underline{V}}_1 + \lambda_2 c_2 \underline{\underline{V}}_2 + \dots + \lambda_n c_n \underline{\underline{V}}_n$$

Eigenvalues, Eigenvectors:

Standard Method (Lecture 4)

Power Method: used to find largest eigenvalue and eigenvector associated with it.

Let $\underline{A}(n \times n)$ be a diagonalizable matrix (n eigenvalues)

$|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|$ and eigenvectors $\underline{v}_1, \dots, \underline{v}_n$ spanning the n -dimensional space. $|\underline{v}_i| = 1 \Rightarrow$ all normalized.

$$\underline{b}_0 = c_1 \underline{v}_1 + c_2 \underline{v}_2 + \dots + c_n \underline{v}_n \quad (c_1 \neq 0)$$

$$\begin{aligned} \underline{A} \underline{b}_0 &= \underline{A} c_1 \underline{v}_1 + \dots + \underline{A} c_n \underline{v}_n \\ &= \lambda_1 c_1 \underline{v}_1 + \dots + \lambda_n c_n \underline{v}_n \end{aligned}$$

$$\underline{A}^k \underline{b}_0 = \lambda_1^k c_1 \underline{v}_1 + \dots + \lambda_n^k c_n \underline{v}_n$$

$$|\lambda_1| > |\lambda_i| \Rightarrow |\lambda_1^k| \gg |\lambda_i^k|$$

$$\lim_{k \rightarrow \infty} \underline{A}^k \underline{b}_0 = \lambda_1^k c_1 \underline{v}_1 = \lambda_1 \underline{v}_1$$

determine λ_1 by applying $\underline{A} \underline{v}_1 = \lambda_1 \underline{v}_1$

Practically: $\underline{b}^{(k+1)} = \frac{\underline{A} \underline{b}^{(k)}}{\|\underline{A} \underline{b}^{(k)}\|}$ ← normalize to keep on the order of 1.

only works if $c_1 \neq 0$, in order to ensure that we take $\underline{b}_0, \underline{b}_0' \perp \underline{b}_0$
if you get one eigenvalue then you are done, if two you got the two largest eigenvalues.

We can manipulate the eigenvalues of $\underline{A}$:

$$-\underline{A} \rightarrow \underline{A}^{-1}, \lambda_i = \frac{1}{\lambda_i}$$

$$-\underline{A} \rightarrow \underline{A} + \beta \underline{I}, \lambda_i = \lambda_i + \beta$$

$$-\underline{v}_1 \underline{v}_1^T = \underline{v}_1 \circ \underline{v}_1: \text{eigenvalues } \|\lambda_1\|^2, 0, 0, \dots, 0$$

- eigenvalue around $\mu \rightarrow$ shift by μ and then inverse $\Rightarrow$ turns smallest to largest
now the eigenvalue closest to μ is closest to zero

- second largest one: shift the largest to ϕ by outer product.

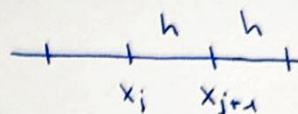
Numerical Derivation: derivative definition: $f'(a) = \lim_{x \rightarrow a} \frac{f(x) - f(a)}{x - a}$

$\Rightarrow$ if $f(x)$ can be evaluated at any point, beware of errors when choosing how close to get to a

Most Common:

Grid: $f(x)$ is evaluated of grid points

$$x_j, \quad j=1, \dots, n, \quad x_{j+1} - x_j = h$$



Forward difference:

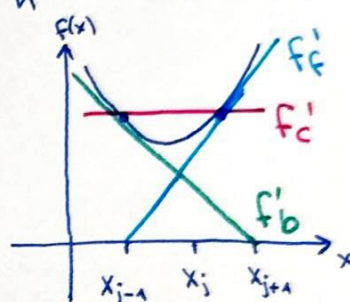
$$f'_f(x) = \frac{f(x_{j+1}) - f(x_j)}{x_{j+1} - x_j} = \frac{f(x_{j+1}) - f(x_j)}{h}$$

Backwards difference:

$$f'_b(x) = \frac{f(x_j) - f(x_{j+1}))}{h}$$

Central difference:

$$f'_c(x) = \frac{f(x_{j+1}) - f(x_{j-1}))}{2h}$$



Error: start from Taylor's series

$$f(x) = \frac{f(x_j)(x-x_j)^0}{0!} + \frac{f'(x_j)(x-x_j)^1}{1!} + \frac{f''(x_j)(x-x_j)^2}{2!} + \dots$$

Forward difference error: $f(x_{j+1}) = f(x_j) + \boxed{f'(x_j)} \overbrace{(x_{j+1} - x_j)}^h + \frac{1}{2} f''(x_j) \overbrace{(x_{j+1} - x_j)^2}^{h^2} + \dots$

$$f'(x_j) = \frac{f(x_{j+1}) - f(x_j)}{h} + \frac{1}{2} f''(x_j) \cdot h + \dots = \frac{f(x_{j+1}) - f(x_j)}{h} + \underbrace{\sigma(h)}_{\text{error}}$$

Same for backwards difference.

Central difference:

$$f(x_{j+1}) = \dots \text{ (same formula) } = f(x_j) + \boxed{f'(x_j)} h + \frac{1}{2} f''(x_j) h^2 + \mathcal{O}(h^3)$$

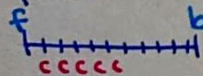
$$f(x_{j-1}) = f(x_j) - \boxed{f'(x_j)} h + \frac{1}{2} f''(x_j) h^2 + \mathcal{O}(h^3)$$

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_{j-1}))}{2h} = 2f'(x_j)h + 0 + \mathcal{O}(h^3)$$

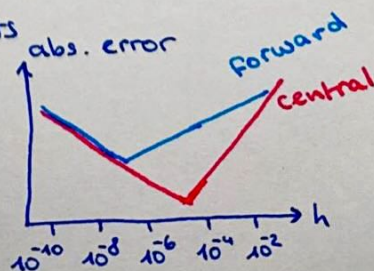
$$f'(x_j) = \frac{f(x_{j+1}) - f(x_{j-1}))}{2h} + \mathcal{O}(h^2) \quad // \text{divided by } h, \text{ central difference is better smaller order of error.}$$

Remarks: - including more distant terms one can get even lower error

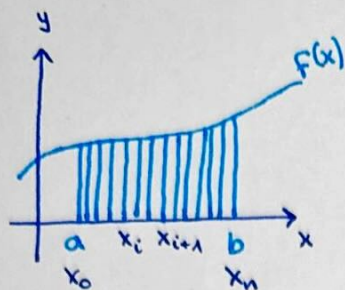
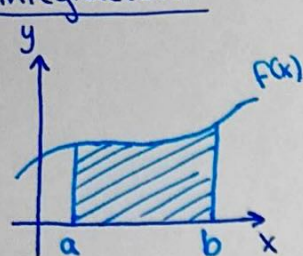
- finite lattice you need to use f and b for end points



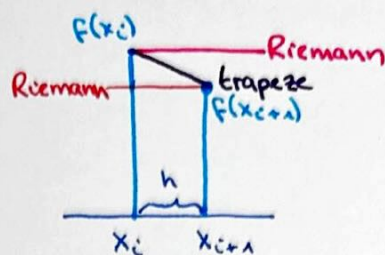
- sometimes higher error due to number of precisions when you increase h



Integration:



$$h = x_{i+1} - x_i$$



$$I_i^{(L)} = x_i \cdot h$$

$$I_i^{(R)} = x_{i+1} \cdot h$$

$$I_i^{(T)} = \frac{x_i + x_{i+1}}{2} \cdot h$$

L: left boundary

R: right boundary

T: both boundary

Methods differ in how we handle boundaries:

$$I^{(L)} = \sum_{i=0}^{n-1} f(x_i) h = f(x_0) h + \sum_{i=1}^{n-1} f(x_i) h$$

$$I^{(R)} = \sum_{i=0}^{n-1} f(x_{i+1}) h = f(x_n) \cdot h + \sum_{i=1}^{n-1} f(x_i) h$$

$$I^{(T)} = \sum_{i=0}^{n-1} \frac{f(x_i) + f(x_{i+1})}{2} h = h \sum_{i=0}^{n-1} \frac{f(x_i)}{2} + h \sum_{i=1}^{n-1} \frac{f(x_i)}{2} = h \frac{f(x_0)}{2} + h \frac{f(x_n)}{2} + h \sum_{i=1}^{n-1} f(x_i)$$

Error Calculation:

Riemann Error: (left) : Taylor series:

constants

$$f(x) \Big|_{x_i}^{x_{i+1}} = f(x_i) + f'(x_i)(x - x_i) + \frac{1}{2} f''(x_i)(x - x_i)^2$$

$$\int_{x_i}^{x_{i+1}} f(x) dx \approx \int_{x_i}^{x_{i+1}} dx \dots = h f(x_i) + \frac{h^2}{2} f'(x_i) + O(h^3)$$

$$\int_{x_i}^{x_{i+1}} f(x) dx = h f(x_i) + O(h^2) \leftarrow \text{one point} \Rightarrow N = \frac{b-a}{2} \text{ total points} \Rightarrow \boxed{\text{Total error } O(h)}$$

Trapeze Error: Taylor at x_i and x_{i+1}

$$f(x) \Big|_{x_i} = f(x_i) + f'(x_i)(x - x_i) \dots$$

$$f(x) \Big|_{x_{i+1}} = f(x_{i+1}) + f'(x_{i+1})(x - x_{i+1}) \dots$$

$$\text{Intermediate point: } y_i = \frac{x_i + x_{i+1}}{2}$$

$$f(x_i) \approx f(y_i) + f'(y_i)(x_i - y_i) + \frac{1}{2} f''(y_i)(x_i - y_i)^2 + \dots$$

$$f(x_{i+1}) \approx f(y_i) + f'(y_i)(x_{i+1} - y_i) + \frac{1}{2} f''(y_i)(x_{i+1} - y_i)^2 + \dots$$

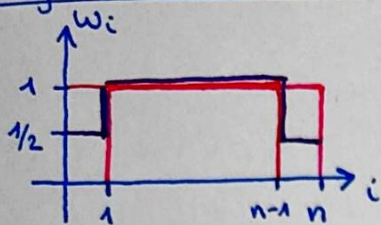
$$\frac{f(x_i) + f(x_{i+1})}{2} = f(y_i) + O(h^2) \quad \text{first order terms cancel each other}$$

$$\int_a^b f(x) dx = \int_a^b (f(y_i) + \underbrace{f'(y_i)(x - y_i)}_{\text{vanishes}} + \dots) dx = h f(y_i) + O(h^3)$$

Total Error: $O(h^2)$

Total error smaller than Riemann when $h < 1$

Higher Order Methods: - wider boundary with different factors



Gauss-Legendre quadrature:

- n : # of points

- x_i : sampling points (free to choose)

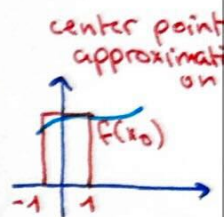
- w_i : weights

$$\int_{-1}^1 f(x) dx \approx \sum_{i=0}^{n-1} w_i f(x_i)$$

$n=1$: $\sum_{i=0}^0 w_i f(x_i) = w_0 f(x_0)$ to be determined x_0, w_0

$f(x) = Ax + b \Rightarrow$ LHS: $\int_{-1}^1 f(x) dx = \int_{-1}^1 (Ax + b) dx = 2b \approx$ RHS: $w_0 (Ax_0 + b)$

$\Rightarrow$ solution $x_0 = 0, w_0 = 2 \quad I \approx 2 \cdot f(x_0)$



$n=2$: 4 unknowns $w_0, w_1, x_0, x_1 \Rightarrow$ 3rd degree polynomial needed

$f(x) = Ax^3 + Bx^2 + Cx + D$

LHS: $\int_{-1}^1 f(x) dx = \int_{-1}^1 (Ax^3 + Bx^2 + Cx + D) dx = \frac{2}{3}B + 2D \approx$ RHS: $w_0(Ax_0^3 + Bx_0^2 + Cx_0 + D) + w_1(Ax_1^3 + Bx_1^2 + Cx_1 + D)$

$$\left. \begin{aligned} w_0 x_0^3 + w_1 x_1^3 &= 0 \\ w_0 x_0^2 + w_1 x_1^2 &= \frac{2}{3} \\ w_0 x_0 + w_1 x_1 &= 0 \\ w_0 + w_1 &= 2 \end{aligned} \right\} \begin{aligned} \text{Solution? non-linear in } x_i, \text{ linear in } w_i \\ w_0 = w_1 = 1 \\ x_0 = \frac{-1}{\sqrt{3}}, x_1 = \frac{1}{\sqrt{3}} \approx 0.577 \end{aligned}$$

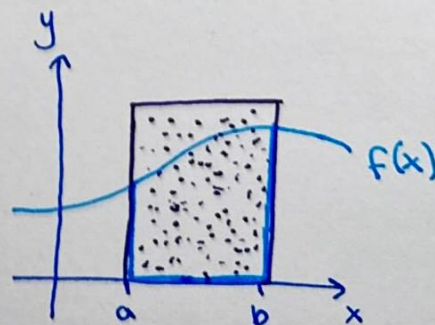
If you only have 2 points choose the ones closer to the boundary exactly $\frac{1}{\sqrt{3}}$ from boundary.

General Case: x_i : are the roots of the degree n Legendre polynomial
 w_i : linear set of equations

Monte Carlo Integration:

- box the function
- calculate area of box A
- generate random points
- measure fraction of the points inside the function: f

$I = f \cdot A$



Introduction to Numerical Algorithms - Lecture 7

Random Numbers:

use np.random instead of built in random

Needed for:

- cryptography
 - stochastic models
 - statistical sampling, ensemble average
- } Simulations such as monte Carlo simulation

Requirements:

cryptography: - uncorrelated, truly random, not needed too frequently (for keys etc.)

Simulations: - uncorrelated, fast generation, reproducible (for debugging), large limit cycle (limit before they start repeating), frequent usage.

True Random Numbers:Sources:

Physical Phenomena: - shot noise (circuits)

- nuclear decay
- atmospheric noise
- lava lamps

Advantages: + truly randomDisadvantages:

- slow generation rate
- expensive to produce
~ kbit/s / 100 Euros.

Pseudorandom Numbers:

- generated by deterministic algorithm but mimics randomness.

Advantages:

+ fast and cheap, deterministic (reproducible) good for debugging, can be parallelized, can be made reliable with proper design

Disadvantages:

- limited period (cycle), sensitive to poor seed or bad parameters can be predictable $\rightarrow$ not secure for cryptography, may include traps.

Multiplicative Congruential Algorithm: (MCA)General Formula:

Let r_j be an integer $\Rightarrow r_{j+1} = (a \cdot r_j + c) \bmod(m)$

Parameters a, c, m :

Historical: $a = 7^5 = 16807$, $m = 2^{31} - 1$, $c = 0$

GCC built in: $a = 1103515245$, $m = 2^{31}$, $c = 12345$

} Good example

Bad Example:

IBM random number generator RANDU

$a = 65539$, $m = 2^{31}$, $c = 0$

Other Algorithms: Permuted Congruential Algorithm (PCA)

on top of MCA, the new number is shifted relative to the old number

$\xleftarrow{-2}$
 $100110 \Rightarrow 011010$ makes a better bitwise randomness

Tests of Randomness: - birthday spacing $\Rightarrow$ should be exponential distr.

- monkey test

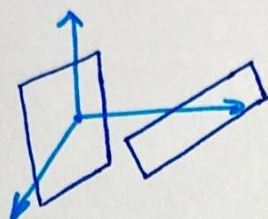
- Fourier Analysis

- moments

- Fill of an n dimensional lattice

- correlation test

- Random walks



$r(x,y,z) \Rightarrow$ RANDU Points are on specific planes
Marsaglia Effect

Seed:

- true random source

- time (default) $\rightarrow$ in server clusters, or parallel cores the time is the same so they will do the same things.

- fixed value $\rightarrow$ saves the values used
 $\hookrightarrow$ seeds must be uncorrelated

Distributions:

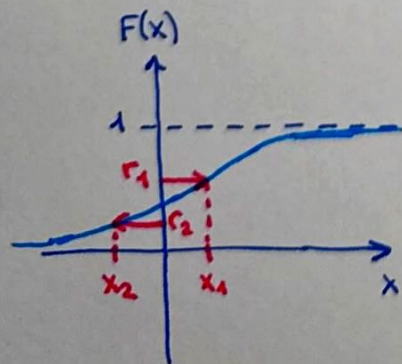
- MCA: generates integer from $[0, n-1]$ with uniform distribution

- $x \in [0, 1[$, $x = \frac{r}{m}$

Probability distribution: $f(x)$

$x \in \mathbb{R}$

Cumulative distribution: $F(x) = \int_{-\infty}^x f(x') dx'$ $F(x)$ monotonous increasing $\in [0, 1]$



Fitting:

- generally means minimizing square error

$$S(\theta) = \sum_i (y_i - f_\theta(x_i))^2$$

θ : set of parameters

$$\theta_{\text{fit}} = \arg \min_{\theta} S(\theta)$$

Example:

Linear fit

$$f(x) = ax + b$$

$$\theta \Leftarrow \{a, b\}$$

If the data includes G_i , uncertainty of point i :

Sum of squared residuals:
$$S(\theta) = \sum_i \left(\frac{y_i - f_\theta(x_i)}{G_i} \right)^2$$

Coefficient of determination: R^2

Measures how well the model explains the variability in data

Residual Sum of Squares:
$$SS_{\text{res}} = \sum_i (y_i - f_\theta(x_i))^2$$

Total sum of squares:
$$SS_{\text{TOT}} = \sum_i (y_i - \bar{y})^2, \quad \bar{y} = \frac{1}{n} \sum_i y_i$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{TOT}}}$$

Common Problems:

- y values of different order of magnitudes

- Exponential $\rightarrow$ solution: semi log

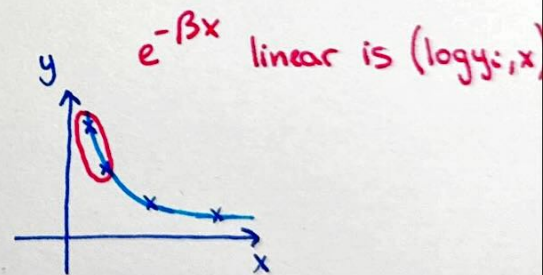
$$y = k e^{\beta x}$$

$$\log y = \log k + \beta x$$

- Power Law $\rightarrow$ solution: double log, (log-log), unitless $\rightarrow$ scale free

$$y = k x^\delta$$

$$\log y = \log k + \delta \log x$$



Error Propagation:

- | | | |
|------------|-------------------------------|---------------------|
| - Sources: | - precision of the instrument | Random |
| | - human error | Random / Systematic |
| | - stochastic system | Random |
| | - environment | Systematic |

Error types:

- random $\Rightarrow$ repeat measurements
- systematic $\Rightarrow$ repeat with changed environment
 $\rightarrow$ difficult to diagnose and recover it

Random Errors:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$$

sample standard deviation:

$$S_x = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

-1 correction factor
of concave nature
of square root.

Standard deviation of the mean:

$\rightarrow$ is reduced by $\sqrt{n}$ compared to σ

$$\sigma_x = \sqrt{\frac{1}{n(n-1)} \sum_{i=1}^n (x_i - \bar{x})^2}$$

- 68% CI : $\bar{x} \pm \sigma$
- 95% CI : $\bar{x} \pm 2\sigma$
- 99.7% CI : $\bar{x} \pm 3\sigma$
- $\vdots$
- 99.9999998% CI $\bar{x} \pm 6\sigma$

Two variable case $f(a,b)$: a, b measured

$$f \approx f^0 + \frac{\partial f}{\partial a} a + \frac{\partial f}{\partial b} b + \dots$$

$$\sigma_f^2 = \left(\frac{\partial f}{\partial a}\right)^2 \sigma_a^2 + \left(\frac{\partial f}{\partial b}\right)^2 \sigma_b^2 + 2 \frac{\partial^2 f}{\partial a \partial b} \sigma_{ab}$$

$\swarrow$ covariance

if a, b independent $\sigma_{ab} = 0$

a, b independent:

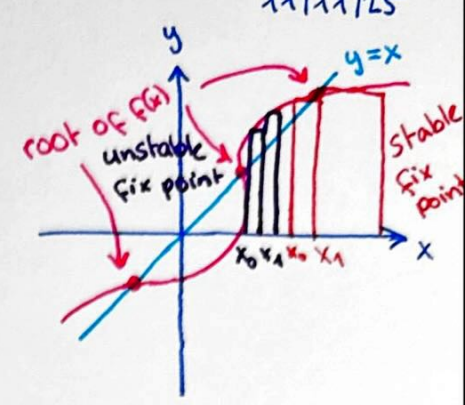
- multiplication by constant $f(a) = c \cdot a \Rightarrow \sigma_f = |c| \cdot \sigma_a$
- add/subtract: $f(a,b) = a \pm b \Rightarrow \sigma_f = \sqrt{\sigma_a^2 + \sigma_b^2}$ $\left| \frac{\partial f}{\partial a} \right| = 1$
 $\uparrow$
 $\frac{1}{\sqrt{n}}$ for the average.
- product: $f(a,b) = a \cdot b$
$$\sigma_f = |f| \sqrt{\frac{\sigma_a^2}{a^2} + \frac{\sigma_b^2}{b^2}}$$
- power: $f(a) = a^\gamma \Rightarrow \sigma_f = |f| \left| \gamma \cdot \frac{\sigma_a}{a} \right|$

Introduction to Numerical Algorithms - Lecture 9

Root Finding:

$f(x)=0$, n dimensions

$f_1(x_1, \dots, x_n)=0$
 $f_2(x_1, \dots, x_n)=0$
 $\vdots$
 $f_n(x_1, \dots, x_n)=0$



Fixed Point Method: (non-linear)

$f(x)=0, g(x)=f(x)+x \Rightarrow \boxed{g(x)=x}$

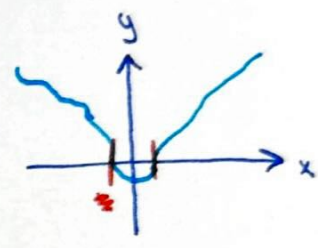
- Two types of fixed points:
- stable $f'(x_i) < 1$
 - unstable $f(x)=0, -f(x)=0 \Rightarrow$ they have the same roots (tűkröze's)
- $g'(x) = -f(x) + x = x - f(x)$
- stable ↔ unstable

Starting Point:

- try to map all suspected area

Higher Dimension: limited instability

stopping precision

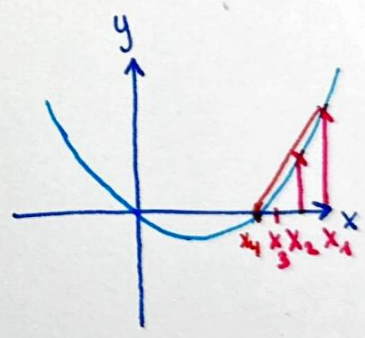


Bisection Method: interval $[x_1, x_2]$ (Bad Method)

if $\text{sign}(f(x_1)) \cdot \text{sign}(f(x_2)) < 0$ then there is at least one root between x_1, x_2

Method: $x_0 = \frac{x_1 + x_2}{2}$ assign x to x_1 or x_2 with the matching sign of $f(x)$

- x_1, x_2 with different signs may be challenging
- middle is a wrong approximation.



False Position Method: (regula falsi)

- predict the root by linear approximation

$$\frac{f(x_3) - f(x_2)}{x_3 - x_2} \approx \frac{f(x_1) - f(x_2)}{x_1 - x_2} \quad \text{set } f(x_3)=0$$

$$x_3 = x_2 - f(x_2) \frac{x_1 - x_2}{f(x_1) - f(x_2)}$$

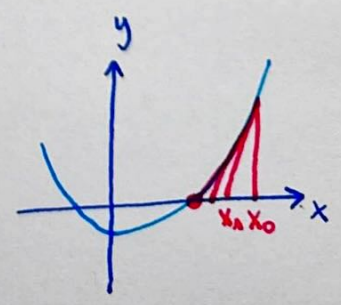
- you can start with any two points
- much faster than bisection method
- works with intervals without a root

Newton Method: single point method which uses f'

Taylor: $0 = f(x_0) + f'(x_0)(x_1 - x_0) + \dots$

$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$

- generally faster than Regula Falsi (RF)
- you need f'



Minimum finding: if you know how to find roots, $f^2(x)$ has minima at the roots.

Gradient Method:

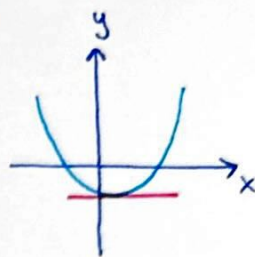
Let $f(x)$ be differentiable and $\gamma > 0$, then $x_{n+1} = x_n - \gamma f'(x_n)$

you can show $f(x_{n+1}) \leq f(x_n)$ provided γ is small enough

from Taylor expansion around the min

$$f(x) \Big|_{x=x_{\min}} \approx f(x_{\min}) + 0 + \frac{1}{2}(x_{\min} - x)^2 f''(x_{\min})$$

we can ~~can~~ calculate γ such that we go to the min in one step



Conjugate Gradient Method:

assuming a parabola $f(x) \approx \beta(x-a)^2$

$x_1, x_2, f(x_1), f(x_2), f'(x_1), f'(x_2)$

$$x_3 = x_2 - \gamma f'(x_2) = x_2 - 2\gamma\beta(x_2 - a)$$

$$x_3 = a \Rightarrow \gamma = \frac{1}{2\beta} = \frac{x_2 - x_1}{f'(x_2) - f'(x_1)}$$

for when you only have the points, not the equation with β



Problems:

local minima: $\Rightarrow$ simulated annealing:

- elementary step: (Monte Carlo chain)
change the configuration

- ergodic: - the whole phase space is reachable
- all possible configurations

- Minimize E : - ΔE (of elementary step) = $E_{\text{new}} - E_{\text{old}}$

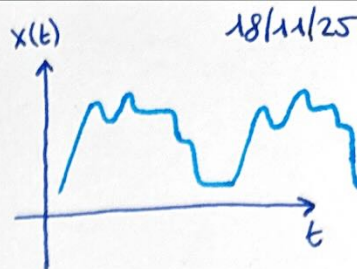
$$\Delta E \begin{cases} < 0, \text{ accept new!} \\ > 0, \text{ accept with probability } \sim \exp\left(-\frac{\Delta E}{T}\right) \end{cases}$$

- decrease T slowly.

Fourier Transform:

What are the frequency components in the signal?

$\Rightarrow$ sum of sine and cosine functions



$\sin(k\omega t)$, $\cos(k\omega t)$ are the basis functions $k \in \mathbb{N}$, $k \in \{0, 1, 2, \dots\}$

$$T = \frac{2\pi}{\omega}$$

$$\int_0^T \sin(k\omega t) \cdot \sin(l\omega t) dt = \frac{T}{2} \delta_{k,l}$$

$$\int_0^T \cos(k\omega t) \cdot \cos(l\omega t) dt = \frac{T}{2} \delta_{k,l}$$

$$\int_0^T \sin(k\omega t) \cdot \cos(l\omega t) dt = 0$$

Fourier Series Expansion: $x(t, T)$ how to express on the basis?

$$x(t, T) = \frac{b_0}{2} + \sum_{k=1}^{\infty} a_k \sin(k\omega t) + b_k \cos(k\omega t)$$

$$b_0 = \frac{2}{T} \int_0^T x(t, T) dt, \quad a_k = \frac{2}{T} \int_0^T x(t, T) \sin(k\omega t) dt$$

$$b_k = \frac{2}{T} \int_0^T x(t, T) \cos(k\omega t) dt$$

The complex formalism:

$$e^{ik\omega t} \text{ comes with } C_k = \frac{a_k}{2} - \frac{b_k}{2i} \quad k \geq 1, \quad \sin x = \frac{e^{ix} - e^{-ix}}{2i}, \quad \cos x = \frac{e^{ix} + e^{-ix}}{2}$$

$$e^{-ik\omega t} \text{ comes with } C_{-k} = \frac{a_k}{2} + \frac{b_k}{2i} \quad k \geq 1, \quad C_0 = \frac{b_0}{2}$$

$$x(t, T) = \sum_{k=-\infty}^{\infty} C_k e^{ik\omega t}$$

$$C_k = \frac{1}{T} \int_0^T x(t, T) e^{-ik\omega t} dt$$

Properties:

- derivative: $\mathcal{F}\left[\frac{d}{dt} x(t)\right] = i\omega \mathcal{F}[x(t)]$

- time shift: $\mathcal{F}[x(t-t_0)] = e^{-i\omega t_0} \mathcal{F}[x(t)]$

Discrete Fourier Transform (DFT): $\{x_n\} = (x_0, x_1, \dots, x_{N-1}) \Rightarrow N$ points

$$X_k = \frac{1}{N} \sum_{n=0}^{N-1} x_n e^{-ik \frac{2\pi}{N} n}, \quad f_k = \frac{1}{N} \sum_{k=0}^{N-1} e^{i \frac{2\pi}{N} nk} \cdot \hat{f}_n$$

Inverse:

$$x_n = \sum_{k=0}^{N-1} X_k e^{ik \frac{2\pi}{N} n}, \quad \hat{f}_n = \sum_{k=0}^{N-1} e^{-i \frac{2\pi}{N} nk} \cdot f_k$$

Sampling frequency: f_s , $\Delta t = \frac{1}{f_s}$, $\Delta f = \frac{f_s}{N}$

$$f_k = k \cdot \Delta f = k \cdot \frac{f_s}{N}, \quad k \in (0, 1, \dots, N-1)$$

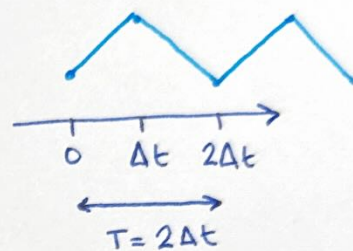
$$f_N = \frac{1}{2\Delta t} = \frac{1}{2} f_s$$

$$\boxed{\frac{N-k}{N} f_s}$$

Nyquist frequency:
maximum frequency you can
resolve

$$0, 1, \dots, \frac{N}{2}, \frac{N}{2} + 1, \dots$$

positive frequency negative frequency



$$e^{i 2\pi f_k} = e^{i 2\pi (1 - \frac{k}{N}) f_s} = e^{-i 2\pi \frac{k}{N} f_s} \Rightarrow \text{negative frequency}$$

Fast Fourier Transform (FFT):

$$X_k = \sum_{n=0}^{N-1} x_n W_N^{nk}, \quad W_N^{nk} = e^{-i \frac{2\pi kn}{N}}$$

Example of $N=8$:

$$W_8^{nk} = \begin{pmatrix} W_8^0 & W_8^0 & W_8^0 & \dots & \dots \\ W_8^0 & W_8^1 & W_8^2 & \dots & \dots \\ W_8^0 & W_8^2 & W_8^4 & W_8^6 & W_8^0 \dots \\ W_8^0 & W_8^3 & W_8^6 & W_8^2 & \dots \\ \vdots & & & & \end{pmatrix}$$

Steps: Cooley-Tukey Algorithm

1. decimation in time
2. compute X_k recursively
3. use the symmetry of the basis

$$(x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7, \dots)$$

$$(0, 2, 4, 6) (1, 3, 5, 7)$$

$$(0, 4) (2, 6) (1, 5) (3, 7)$$

$$X_k = \sum_{n=0}^{N-1} x_n W_N^{nk} = \sum_{m=0}^{N/2-1} x_{2m} W_{N/2}^{km} + W_N^k \sum_{m=0}^{N/2-1} x_{2m+1} W_{N/2}^{km} = E_k + W_N^k O_k$$

$$X_{\frac{K+N}{2}} = E_k - W_N^k O_k$$

$$X_k = x_0 + x_1 W_2^k + W_N^k (x_2 + x_0 W_2^k) + W_8^k ((x_1 + x_3 W_2^k) + W_N^k (x_3 + x_7 W_2^k))$$

$$O(N^2) \rightarrow O(N \log N)$$

Boundary Value problems:

$$\frac{dx}{dt} = f(x, t)$$

$$x(t_0) = x_0$$

what is $x(t) = ?$

$$\begin{aligned} dx &\rightarrow \Delta x \\ dt &\rightarrow \Delta t \end{aligned}$$

Euler Method:

$$x_{n+1} = x_n + f(x_n, t_n) \Delta t$$

you can use arbitrary units, computer does not care about units

 $\Rightarrow$ dimensionlessStability of the Euler Method:

$$f(x) = -kx, \quad \frac{dx}{dt} = -kx \quad x_{n+1} = x_n(1 - k\Delta t) \text{ converges if } |1 - k\Delta t| < 1$$

if $x \in \mathbb{R}^n$ is a vector: - each component separately

Stability: eigensystem, $|\lambda|$ is largest eigenvalue $\Rightarrow |1 + \lambda\Delta t| < 1$
 for stability lambda between zero and lower bound from equation

$$\text{Implicit Euler: } x_{n+1} = x_n + f(x_{n+1}, t_{n+1}) \Delta t \quad \text{solve for } x_{n+1}$$

$$f(x) = -kx, \quad k > 0, \quad x_{n+1} = \frac{x_n}{1 + k\Delta t}$$

implicit Euler is stable if $k > 0$ Error for both methods: $\mathcal{O}(\Delta t)$ Runge-Kutta: RK4: $h \equiv \Delta t$

$$x_{n+1} = x_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad t_{n+1} = t_n + h$$

$$k_1 = f(x_n, t_n)$$

slope at the beginning

$$k_2 = f(x_n + h \frac{k_1}{2}, t_n + \frac{h}{2})$$

mid way check using Euler estimate.

$$k_3 = f(x_n + h \frac{k_2}{2}, t_n + \frac{h}{2})$$

improved mid way

$$k_4 = f(x_n + h k_3, t_n + h)$$

end point.

for large Δt use Euler

	Implicit Euler	RK4
Error:	$\mathcal{O}(h)$	$\mathcal{O}(h^5)$
Stability:	always	limit

Second order differential Equations (Newton's Equation of Motion):

$$\ddot{X} = \frac{F}{m} = f(x, t), \quad x(t=0) = x_0, \quad v(t=0) = v_0$$

Euler method: $x_{n+1} = x_n + v_n \Delta t$
 $v_{n+1} = v_n + f(x_n) \Delta t$

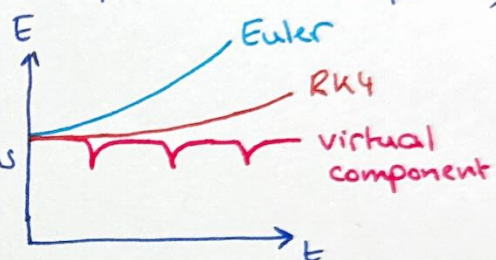
(don't use this its not stable)

Implicit Euler: $v_{n+1} = v_n + f(x_n) \Delta t$
 $x_{n+1} = x_n + \frac{1}{2} v_{n+1} \Delta t$

Symplectic Integrator: keeps the energy constant (plus virtual component)

Higher order Methods:

RK4? $\Rightarrow$ not symplectic $\Rightarrow$ used in Chaotic systems



Standard Method: (for molecular dynamics)

Velocity Verlet: (symplectic integrator)

$$(1) \quad x(t + \Delta t) = x(t) + v(t) \Delta t + \frac{1}{2} a(t) \Delta t^2$$

$$(2) \quad v(t + \Delta t) = v(t) + \frac{a(t) + a(t + \Delta t)}{2} \Delta t$$

Algorithm:

1. $x(t + \Delta t) = \dots$ (1)

2. $v(t + \frac{\Delta t}{2}) = v(t) + \frac{a(t)}{2} \Delta t$

3. get $a(t + \Delta t)$ from $x(t + \Delta t)$

4. $v(t + \Delta t) = v(t + \frac{\Delta t}{2}) + \frac{a(t + \Delta t)}{2} \Delta t$

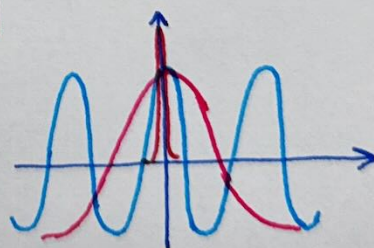
Fourier Transform:

$$\mathcal{F}[\delta(t)] = \frac{1}{\sqrt{2\pi}}$$

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} d\omega e^{-i\omega t} \int_{-\infty}^{\infty} x(t') e^{i\omega t'} dt'$$

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \frac{1}{2\pi} x(t') e^{i\omega(t+t')} dt' d\omega = \underbrace{\frac{1}{2\pi} \int_{-\infty}^{\infty} x(t') dt'}_{\delta(t-t')} \underbrace{\int_{-\infty}^{\infty} e^{i\omega(t-t')} d\omega}_{\delta(t-t')}$$

$$x(t) = \int_{-\infty}^{\infty} x(t') \delta(t'-t) dt'$$



Fourier transform
of the Dirac
Delta

Fourier Transform:

$$F(\omega) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt, \quad f(t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} F(\omega) e^{i\omega t} d\omega$$

Derivation:

$$\begin{aligned} \frac{d}{dt} f(t) &= \frac{d}{dt} \left[\frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} F(\omega) e^{i\omega t} d\omega \right] = \\ &= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \frac{d}{dt} e^{i\omega t} F(\omega) d\omega = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} i\omega F(\omega) e^{i\omega t} d\omega \end{aligned}$$

$$F\left[\frac{d}{dt} f(t)\right] = i\omega F(\omega)$$

Example: $\frac{dx}{dt} + kx = g(t)$ /FT

damping $\swarrow$ $\nwarrow$ *drive*

$$i\omega \hat{x}(\omega) + k \hat{x}(\omega) = \hat{g}(\omega) \rightarrow \hat{x} = \frac{\hat{g}(\omega)}{i\omega + k}$$

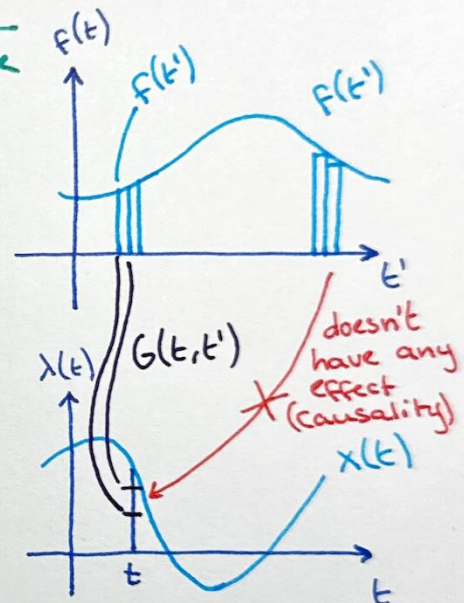
Green's Function:

$f(t)$ input (drive) $x(t)$ output

Linear system: $\left. \begin{aligned} f_1(t) + f_2(t) &\rightarrow x_1(t) + x_2(t) \\ \lambda f(t) &\rightarrow \lambda x(t) \end{aligned} \right\}$ linear system

$$f(t) \rightarrow x(t) = ?$$

$$i) \quad x(t) = \int_{-\infty}^{\infty} G(t, t') f(t') dt' \quad \text{Linearity}$$



ii) Causality: future events do not affect the present (if temporal)
if it is space derivation then you do not have this.

$$x(t) = \int_{-\infty}^{\infty} G(t, t') f(t') dt'$$

iii) Time invariance: doesn't matter if you do the experiment in the morning or afternoon, only time difference matters

$\tau = t - t'$ time difference

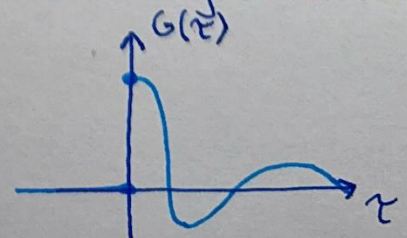
$$x(t) = \int_0^{\infty} G'(\tau) f(t - \tau) d\tau \quad (\text{different } G \text{ not derivative})$$

$$G(\tau) = \Theta(\tau) G'(\tau)$$

$\nwarrow$ causality

$$x(t) = \int_{-\infty}^{\infty} G(\tau) f(t - \tau) d\tau \quad \text{convolution}$$

cable oscillating demonstration



Calculate the Green's Function: put in Dirac Delta

$$f(t) \equiv \delta(t)$$

$$G(t) = \int_{-\infty}^{\infty} G(\tau) \delta(t-\tau) d\tau$$

Green's function is the response to the dirac delta drive

Solve the homogenous system with

1st order diff. equ. $G(t=0)=1$

2nd order diff. equ. $G(t=0)=0, G'(t=0)=1$

Fourier Space Example: damped harmonic oscillator

$$\ddot{x} + 2\lambda \dot{x} + \omega_0^2 = f(t) \quad / \text{FT}$$

$$-w^2 \hat{x} + 2\lambda i w \hat{x} + \omega_0^2 = \hat{f}(w) \quad \Rightarrow \quad \hat{x} = \frac{\hat{f}(w)}{-w^2 + 2\lambda i w + \omega_0^2}$$

Convolution: $\hat{f}[(f * g)] = \hat{f}\left[\int_{-\infty}^{\infty} f(\tau) g(t-\tau) d\tau\right] = \sqrt{2\pi} \hat{f}(w) \hat{g}(w)$

Green's Function:

$$\hat{x} = \sqrt{2\pi} \hat{G}(w) \hat{f}(w) \quad \Rightarrow \quad G(w) = \frac{1}{\sqrt{2\pi}} \frac{1}{-w^2 + 2\lambda i w + \omega_0^2}$$

$$\boxed{\hat{f}[\delta(t)] = \frac{1}{\sqrt{2\pi}}}$$

Damped harmonic oscillator:

$$\hat{G}(w) = |\hat{G}(w)| e^{i\gamma} \quad \text{Euler form:} \quad \tan \gamma = - \frac{2\lambda w}{\omega_0^2 - w^2}$$

$$|\hat{G}(w)| = \frac{1}{2\pi \sqrt{(\omega_0^2 - w^2)^2 + 4\lambda^2 w^2}}$$

$$\omega_R = \sqrt{\omega_0^2 - 2\lambda^2}$$

